

DOCUMENTACIÓN - MikeRides-JSF-Hibernate

Repositorio GitHub: <https://github.com/SikeMike/MikeRides-JSF-Hibernate>

Video explicativo: <https://photos.app.goo.gl/G4MQ2ZkcucFmF8BeA>

En este documento se va a explicar en términos generales el proceso que se ha llevado a cabo para realizar las modificaciones al proyecto de RidesComplete, y generar uno nuevo en el que se implemente el framework de JSF y Hibernate.

El desarrollo se puede dividir en dos etapas o fases (aunque, en realidad, ambas etapas se han desarrollado la una de la mano de la otra:

1. Implementación de la Interfaz haciendo uso de JSF, sustituyendo la interfaz gráfica original que hacía uso de Java Swing.
2. Implementación de la capa de base de datos, reemplazando la antigua base de datos ObjectDB por un modelo relacional gestionado con Hibernate, otorgando robustez.

En relación a las funcionalidades que se han desarrollado:

- Se ha desarrollado el caso de uso de CreateRide
- Se ha desarrollado el caso de uso de QueryRide
- Se ha desarrollado el caso de uso adicional de Login
- Se ha desarrollado el caso de uso adicional de Registro.

PROCESO SEGUIDO:

A la hora de realizar el desarrollo de este proyecto, se ha hecho de la siguiente manera.

A pesar de que, como estaba estipulado en uno de los dos enunciados relacionados con este proyecto, se nos recomendaba primero importar la lógica de negocio y el modelo de dominio directamente a la estructura del proyecto mediante una importación, tras intentar esta vía, como desarrollador del proyecto, opté por tomar otra decisión.

Siguiendo un poco el curso de acciones que se aprenden en los laboratorios guiados de JSF1, JSF2, JSF3 y Hibernate1, Hibernate2, Hibernate3, se decidió seguir los siguientes pasos:

Primero, al igual que se indica en el laboratorio JSF1, se creó un proyecto Maven desde cero. Se realizaron las configuraciones necesarias en ficheros clave e importaciones requeridas para tener un proyecto inicial funcional sin errores.

Para ello, primero se definió el pom.xml, con todas las dependencias requeridas, y como punto de entrada del proyecto, se definió Index.xhtml para poder verificar que se habían realizado correctamente todos los pasos para el funcionamiento del framework, y que era capaz de renderizar el Index. Esto se estipula (junto con configuración adicional) en el fichero de webapp/WEB-INF/web.xml. Y también, como requerimiento, se insertó el archivo webapp/WEB-INF/beans.xml, en el que se le especifica dónde debe de buscar los ficheros "bean".

Una vez que se vió que el renderizado funcionaba correctamente, se añadió un Register.xhtml y un Login.xhtml (exentos de funcionalidad) para poder testear el “enrutamiento”, o dicho de otra forma, la conexión entre distintas rutas. Para ello, se tuvo que implementar el webapp/WEB-INF/faces-config.xml, en el cual se especifica la conexiones entre los distintos archivos.

En algunos casos se usará este fichero para determinar a qué otra ruta se debe de ir, y en otros casos se usará directamente un string (por ejemplo, `"Index?faces-redirect=true"` que permita redireccionar directamente.

A continuación, se realizaron las adaptaciones al proyecto para que pudiera hacer uso de Hibernate.

Se añadió el fichero de configuración de hibernate persistence.xml (con información relacionada a la conexión con la base de datos albergada en MariaDB).

Además, se añadió la clase JPAUtil.java, encargada de obtener la instancia de “Entity Manager Factory” que a su vez nos permitirá interactuar con las tablas de la base de datos mediante queries.

Por último, se creó la base de datos “mikerides” en el entorno de MariaDB.

A continuación, se inició con el desarrollo del caso de uso de “Register”.

Para ello, tuvo que implementarse el bean RegisterBean.java, el cual se encarga de obtener los datos del formulario que se muestra en Register.xhtml (el cual incluye los datos de *username* y *password*). Para poder desarrollar el resto del caso de uso, cogiendo como base la implementación realizada en el proyecto proporcionado “RidesComplete”, se implementaron las siguientes clases (pero solo conteniendo las funciones relevantes y estrictamente necesarias para el funcionamiento de los casos de uso implementados en este proyecto):

- Driver.java (con sus pertinentes etiquetas como `@Entity` o `@Id` para su correcta gestión y almacenamiento en la base de datos)
- BLFacade.java como interfaz
- BLFacadeImplementation.java como implementación de la interfaz
- DataAccessHibernate.java como clase que se encarga de realizar las transacciones con la base de datos, y enviar los resultados a la capa de lógica de negocio.

El siguiente caso de uso que se realizó fue el de “Login”, que, además de requerir su bean pertinente LoginBean.java, no requirió de demasiado desarrollo adicional.

A continuación se añadió el DriverMenu.xhtml en el cual se añadirían botones para acceder a portales con las siguientes dos funcionalidades a desarrollar (especificando, por supuesto, las rutas en faces-config.xml).

El siguiente caso de uso que se realizó fue el de “Create Ride”. Para implementar este caso de uso, se tuvo que implementar una nueva clase de dominio Ride.java, y se añadió un atributo (tanto en esta clase como en la de Driver.java) para establecer una relación entre ambas entidades de cardinalidad 1..N. Una vez actualizado el dominio, se implementó CreateRide.xhtml y CreateRideBean.java. Al añadir estos dos, puesto que se hace uso de un calendario específico de la librería de PrimeFaces, se tuvieron que hacer algunos cambios en ficheros de configuración como pom.xml, web.xml...

Por último, se realizó el caso de uso de “Query Rides”. Para implementar este caso, se añadieron QueryRides.xhtml y QueryRidesBeans.xhtml, y en este caso específico se hizo algo dificultoso averiguar cómo realizar la implementación en la capa de presentación para poder mostrar los “rides” obtenidos en pantalla.